

Nota de Estudo 1 - Criptografia

A) Introdução

Técnicas criptográficas permitem que um remetente disfarce dados de forma que um intruso não possa obter nenhuma informação dos dados interceptados. O receptor, é claro, deve ser capaz de recuperar os dados originais a partir dos dados disfarçados.

A

Suponha que Alice queira enviar uma mensagem para Bob. A mensagem de Alice em sua forma original (por exemplo, "Bob, eu te amo, Alice") é conhecida como texto simples ou texto claro. Alice criptografa sua mensagem de texto simples usando um algoritmo de criptografia para que a mensagem criptografada, conhecida como texto cifrado, pareça ininteligível para qualquer intruso.

Em muitos sistemas criptográficos modernos, incluindo aqueles usados na internet, a técnica de criptografia em si é conhecida, padronizada e disponível para todos (RFC 1321; RFC 3447; RFC 2420; NIST 2001]. Se todos conhecem o método de codificação de dados, então deve haver alguma informação secreta que impeça um intruso de decifrar os dados transmitidos e é aqui que entram as chaves.

Na Figura 1, Alice fornece uma chave, K_A , uma sequência de números ou caracteres, como entrada para o algoritmo de criptografia. O algoritmo de criptografia recebe a chave e a mensagem de texto simples, m , como entrada e produz o texto cifrado como saída. A notação $K_A(m)$ refere-se à forma de texto cifrado (criptografada usando a chave K_A) da mensagem de texto simples, m . O algoritmo de criptografia real que usa a chave K_A será evidente a partir do contexto. Da mesma forma, Bob fornecerá uma chave, K_B , ao algoritmo de decifração que recebe o texto cifrado e a chave de Bob como entrada e produz o texto simples original como saída. Ou seja, se Bob recebe uma mensagem criptografada $K_A(m)$, ele a decifra calculando $K_B(K_A(m)) = m$. Em sistemas de chaves simétricas, as chaves de Alice e Bob são idênticas e secretas. Em sistemas de chaves públicas, um par de chaves é usado. Uma das chaves é conhecida tanto por Bob quanto por Alice (e pelo mundo inteiro). A outra chave é conhecida apenas por Bob ou Alice (mas não por ambos).

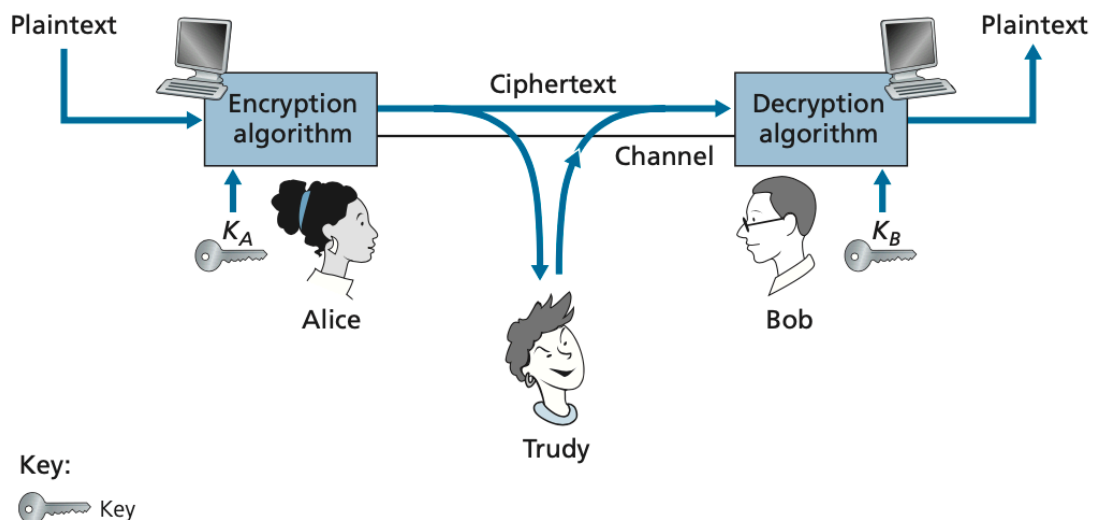


Figura 1

Todos os algoritmos criptográficos envolvem a substituição de uma coisa por outra, por exemplo, pegar um pedaço de texto simples e, em seguida, computar e substituir o texto cifrado apropriado para criar a mensagem criptografada.

Chaves e o princípio de Kerckhoff.

Se um adversário que esteja espionando conhece o algoritmo para descriptografar bem como a chave k compartilhada pelas duas partes que se comunicam, então esse adversário será capaz de decifrar quaisquer textos cifrados transmitidos por essas partes. É por esse motivo que as partes que se comunicam devem compartilhar a chave k com segurança e manter k completamente secreto de todos os outros. Talvez eles devessem manter o algoritmo de decifração em segredo também? Aliás, não seria melhor para eles manterem todos os detalhes do esquema de criptografia em segredo?

No final do século XIX, Auguste Kerckhoff argumentou o oposto em um artigo que ele escreveu elucidando vários princípios de design para cifras militares. Um dos mais importantes deles, agora conhecido simplesmente como princípio de Kerckhoff, era:

O método de cifra não deve ser secreto e deve poder cair nas mãos do inimigo sem inconvenientes.

Ou seja, um esquema de criptografia deve ser projetado para ser seguro mesmo que um espião conheça todos os detalhes do esquema, desde que o invasor não saiba a chave que está sendo usada. Em outras palavras, a segurança não deve depender do segredo do esquema de criptografia; em vez disso, o princípio de Kerckhoff exige que a segurança dependa exclusivamente do sigilo da chave. Há três argumentos principais a favor do princípio de Kerckhoff. O primeiro é que é significativamente mais fácil para as partes manter o sigilo de uma chave curta do que manter em segredo o algoritmo (mais complicado) que estão usando. Isso é especialmente verdadeiro se imaginarmos o uso de criptografia para proteger a comunicação entre todos os pares de funcionários em alguma organização. A menos que cada par de partes use seu próprio algoritmo exclusivo, algumas partes saberão o algoritmo usado por outras. Informações sobre o algoritmo de criptografia podem ser vazadas por um desses funcionários (por exemplo, após ser demitido) ou obtidas por um invasor usando engenharia reversa. Em suma, é simplesmente irrealista presumir que o algoritmo de criptografia permanecerá secreto.

Em segundo lugar, caso as informações secretas compartilhadas pelas partes honestas sejam expostas, será muito mais fácil para elas alterar uma chave do que substituir um esquema de criptografia. (Considere atualizar um arquivo em vez de instalar um novo programa.) Além disso, é relativamente trivial gerar um novo segredo aleatório, enquanto seria uma tarefa enorme projetar um novo esquema de criptografia.

Finalmente, para implantação em larga escala, é significativamente mais fácil para todos os usuários confiarem no mesmo algoritmo/software de criptografia (com chaves diferentes) do que usarem seu próprio algoritmo personalizado. (Isso é verdade mesmo para um único usuário que esteja se comunicando com várias partes diferentes.) De fato, é desejável que os esquemas de criptografia sejam padronizados para que (1) a compatibilidade seja garantida por padrão e (2) os usuários utilizem um esquema de criptografia que tenha passado por escrutínio público e no qual nenhuma fraqueza tenha sido encontrada.

Atualmente, o princípio de Kerckhoff é entendido como a defesa de que os projetos criptográficos sejam tornados completamente públicos, em forte contraste com a noção de “segurança pela obscuridade”, que sugere que manter algoritmos secretos melhora a segurança. É muito perigoso usar um algoritmo proprietário, “caseiro” (ou seja, um algoritmo não padronizado, projetado em segredo por alguma empresa). Em contraste, projetos publicados passam por revisão pública e, portanto, provavelmente são mais fortes. Muitos anos de experiência demonstraram que é muito difícil construir bons esquemas criptográficos. Portanto, nossa confiança na segurança de um esquema é muito maior se ele tiver sido extensivamente estudado (por especialistas que não os projetistas do esquema) e nenhuma fraqueza tiver sido encontrada. Por mais simples e óbvio que pareça, o princípio do projeto criptográfico aberto (ou seja, o princípio de Kerckhoff) tem sido ignorado repetidamente, com resultados desastrosos. Felizmente, hoje existem sistemas criptográficos seguros, padronizados e amplamente disponíveis o suficiente para que não haja razão para usar qualquer outra coisa.

Antes de estudar um sistema criptográfico moderno baseado em chaves, vamos primeiro nos familiarizar com um algoritmo de chave simétrica muito antigo e muito simples, atribuído a Júlio César, conhecido como cifra de César. A cifra de César funciona pegando cada letra da mensagem em texto simples e substituindo a letra que está k letras depois. Por exemplo, se $k = 3$, então a letra a em texto simples se torna d em texto cifrado; b em texto simples se torna e em texto cifrado, e assim por diante. Aqui, o valor de k serve como chave. Como exemplo, a mensagem em texto simples “bob, eu te amo. Alice” se torna “ere, l oryh brx. dolfh” em texto cifrado. Embora o texto cifrado pareça sem sentido, não demoraria muito para decifrar o código se você soubesse que a cifra de César estava sendo usada, pois há apenas 25 valores-chave possíveis.

Uma melhoria em relação à cifra de César é a cifra monoalfabética, que também substitui uma letra do alfabeto por outra. No entanto, em vez de substituir de acordo com um padrão regular (por exemplo, substituição com um deslocamento de k para todas as letras), qualquer letra pode ser substituída por qualquer outra letra, desde que cada letra tenha uma letra substituta única, e vice-versa.

Plaintext letter:	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Ciphertext letter:	m	n	b	v	c	x	z	a	s	d	f	g	h	j	k	l	p	o	i	u	y	t	r	e	w	q

Figura 2

Com base na Figura 2, a mensagem em texto simples “bob, eu te amo. Alice” se torna “nkn, s gktc wky. Mgsbc”. Assim, como no caso da cifra de César, isso parece sem sentido. Uma cifra monoalfabética também parece ser melhor do que a cifra de César, pois há $26!$ pares possíveis de letras em vez de 25 pares possíveis. Uma abordagem de força bruta, tentando todos os 1026 pares possíveis, exigiria muito mais trabalho para ser uma maneira viável de quebrar o algoritmo de criptografia e decodificar a mensagem. No entanto, por meio de análise estatística da linguagem de texto simples, por exemplo, sabendo que as letras “e” e “t” são as letras mais frequentes em textos típicos em inglês (representando 13% e 9% das ocorrências de letras), e sabendo que ocorrências específicas de duas e três letras aparecem juntas com bastante frequência (por exemplo, “in”, “it”, “the”, “ion”, “ing” e assim por diante), torna

relativamente fácil quebrar esse código. Se o intruso tiver algum conhecimento sobre o possível conteúdo da mensagem, será ainda mais fácil quebrar o código.

Ao considerar a facilidade com que Trudy quebraria o esquema de criptografia de Bob e Alice, podemos distinguir três cenários diferentes, dependendo das informações que o intruso possui:

- Ataque somente ao texto cifrado: em alguns casos, o intruso pode ter acesso apenas ao texto cifrado interceptado, sem informações precisas sobre o conteúdo da mensagem em texto simples. A análise estatística pode auxiliar em um ataque somente ao texto cifrado em um esquema de criptografia.
- Ataque de texto simples conhecido. Vimos acima que, se Trudy, de alguma forma, soubesse com certeza que "bob" e "alice" apareciam na mensagem de texto cifrado, ela poderia ter determinado os pares (texto simples, texto cifrado) para as letras a, l, i, c, e, b e o. Trudy também pode ter tido a sorte de registrar todas as transmissões em texto cifrado e, em seguida, encontrado a versão decifrada de Bob de uma das transmissões, rabiscada em um pedaço de papel.
- Ataque de texto simples escolhido: em um ataque de texto simples escolhido, o invasor consegue escolher a mensagem de texto simples e obter sua forma cifrada correspondente. Para técnicas de criptografia mais sofisticadas, um ataque de texto simples escolhido não significa necessariamente que a técnica de criptografia possa ser quebrada.

Quinhentos anos atrás, técnicas que aprimoraram a criptografia monoalfabética, conhecidas como criptografia polialfabética, foram inventadas. A ideia por trás da criptografia polialfabética é usar múltiplas cifras monoalfabéticas, com uma cifra monoalfabética específica para codificar uma letra em uma posição específica na mensagem de texto simples. Assim, a mesma letra, aparecendo em posições diferentes na mensagem de texto simples, pode ser codificada de forma diferente. Um exemplo de um esquema de criptografia polialfabética é mostrado na Figura 3. Ele tem duas cifras de César (com $k = 5$ e $k = 19$), mostradas como linhas. Podemos escolher usar essas duas cifras de César, C_1 e C_2 , no padrão repetitivo C_1, C_2, C_2, C_1, C_2 . Ou seja, a primeira letra do texto simples deve ser codificada usando C_1 , a segunda e a terceira usando C_2 , a quarta usando C_1 e a quinta usando C_2 . O padrão então se repete, com a sexta letra sendo codificada usando C_1 , a sétima com C_2 e assim por diante. A mensagem em texto simples "bob, eu te amo." é, portanto, criptografada como "ghu, n etox dhz". Observe que o primeiro "b" na mensagem em texto simples é criptografado usando C_1 , enquanto o segundo "b" é criptografado usando C_2 . Neste exemplo, a "chave" de criptografia e descryptografia é o conhecimento das duas chaves de César ($k = 5, k = 19$) e o padrão C_1, C_2, C_2, C_1, C_2 .

Plaintext letter:	a b c d e f g h i j k l m n o p q r s t u v w x y z
$C_1(k = 5)$:	f g h i j k l m n o p q r s t u v w x y z a b c d e
$C_2(k = 19)$:	t u v w x y z a b c d e f g h i j k l m n o p q r s

Figura 3

B) Princípios da Criptografia Moderna

Como deve ter ficado claro nos exemplos anteriores, a seção anterior, a criptografia foi historicamente mais uma arte do que uma ciência. Os esquemas eram projetados de forma ad hoc e avaliados com base em sua complexidade ou inteligência percebida. Um esquema seria analisado para verificar se algum ataque poderia ser encontrado; em caso afirmativo, o esquema seria "corrigido" para impedir esse ataque, e o processo seria repetido. Embora possa ter havido consenso de que alguns esquemas não eram seguros (como evidenciado por um ataque especialmente prejudicial), não havia uma noção consensual sobre quais requisitos um esquema "seguro" deveria satisfazer, e nenhuma maneira de fornecer evidências de que qualquer esquema específico era seguro.

Nas últimas décadas, a criptografia se tornou mais uma ciência. Os esquemas agora são desenvolvidos e analisados de forma mais sistemática com o objetivo final de fornecer uma prova rigorosa de que uma determinada construção é segura. Para articular tais provas, primeiro precisamos de definições formais que definam exatamente o que "seguro" significa; Tais definições são úteis e interessantes por si só. Como se constata, a maioria das provas criptográficas se baseia em suposições atualmente não comprovadas sobre a dificuldade algorítmica de certos problemas matemáticos; tais suposições devem ser explícitas e declaradas com precisão. A ênfase em definições, suposições e provas distingue a criptografia moderna da criptografia clássica; discutiremos esses três princípios a continuação.

Princípio 1: Uma das principais contribuições da criptografia moderna foi o reconhecimento de que definições formais de segurança são essenciais para o projeto, estudo, avaliação e uso adequados de primitivas criptográficas. Em termos simples: Se você não entende o que deseja alcançar, como poderá saber quando (ou se) o alcançou?

Definições formais proporcionam esse entendimento, fornecendo uma descrição clara de quais ameaças estão no escopo e quais garantias de segurança são desejadas. Assim, definições podem ajudar a orientar o projeto de esquemas criptográficos. De fato, é muito melhor formalizar o que é necessário antes do início do processo de projeto, do que elaborar uma definição post facto após a conclusão do projeto.

Esta última abordagem corre o risco de a fase de projeto terminar quando a paciência dos projetistas se esgotar (em vez de quando o objetivo for alcançado), ou pode resultar em uma construção que alcança mais do que o necessário em detrimento da eficiência.

As definições também oferecem uma maneira de avaliar e analisar o que é construído. Com uma definição estabelecida, pode-se estudar um esquema proposto para verificar se ele alcança as garantias desejadas; em alguns casos, pode-se até provar que uma determinada construção é segura, mostrando que ela atende à definição. Por outro lado, as definições podem ser usadas para demonstrar conclusivamente que um determinado esquema não é seguro, na medida em que não satisfaz a definição. Em particular, observe que os ataques na seção anterior não demonstram automaticamente que qualquer um dos esquemas mostrados ali é "inseguro". Por exemplo, o ataque à cifra de Vigenère presumiu que um texto em inglês suficientemente longo estava sendo criptografado, mas a cifra de Vigenère poderia ser "segura" se um texto curto em inglês, ou texto compactado (que terá frequências de letras aproximadamente uniformes), fosse criptografado? É difícil afirmar sem uma definição formal estabelecida.

As definições permitem uma comparação significativa de esquemas. Como veremos, pode haver várias maneiras (válidas) de definir segurança; o "certo" depende do contexto em que um esquema é usado. Um esquema que satisfaz uma definição mais fraca pode ser mais eficiente do que outro que satisfaz uma definição mais forte; com definições precisas, podemos avaliar adequadamente as compensações entre os dois esquemas. Da mesma forma, as definições permitem o uso seguro de esquemas. Considere a questão de decidir qual esquema de criptografia usar para uma aplicação maior. Uma maneira sensata de abordar o problema é primeiro entender qual noção de segurança é necessária para aquela aplicação e então encontrar um esquema de criptografia que satisfaça essa noção. Um benefício colateral dessa abordagem é a modularidade: um projetista pode "trocar" um esquema de criptografia e substituí-lo por outro (que também satisfaça a definição necessária de segurança) sem ter que se preocupar em afetar a segurança da aplicação como um todo.

Escrever uma definição formal força a pessoa a pensar sobre o que é essencial para o problema em questão e quais propriedades são estranhas. Passar pelo processo frequentemente revela sutilezas do problema que não eram óbvias à primeira vista. Ilustraremos isso a seguir para o caso da criptografia.

Um exemplo - criptografia segura: embora exista a uma definição formal de criptografia segura (como visto em sala de aula) descrevemos aqui informalmente o que tal definição deve capturar. Em geral, uma definição de segurança tem dois componentes: uma garantia de segurança (ou, do ponto de vista do invasor, o que constitui um ataque bem-sucedido ao esquema) e um modelo de ameaça. A garantia de segurança define o que o esquema pretende impedir que o invasor faça, enquanto o modelo de ameaça descreve o poder do adversário, ou seja, quais ações o invasor é considerado capaz de realizar.

O que um esquema de criptografia seguro deve garantir? Aqui estão algumas reflexões:

- Deve ser impossível para um invasor recuperar a chave. Observamos anteriormente que, se um invasor puder determinar a chave compartilhada por duas partes usando algum esquema, então esse esquema não pode ser seguro. No entanto, é fácil criar esquemas para os quais a recuperação da chave é impossível, mas o esquema é flagrantemente inseguro. Considere, por exemplo, o esquema em que $Enc_k(m) = m$. O texto cifrado não vaza nenhuma informação sobre a chave (e, portanto, a chave não pode ser recuperada se for longa o suficiente), mas a mensagem é enviada sem criptografia! Vemos, portanto, que a incapacidade de recuperar a chave não é suficiente para a segurança. Isso faz sentido: o objetivo da criptografia é proteger a mensagem; a chave é um meio para isso, mas, em si, não é importante.
- Deveria ser impossível para um invasor recuperar todo o texto simples a partir do texto cifrado. Esta definição é melhor, mas ainda está longe de ser satisfatória. Em particular, esta definição consideraria um esquema de criptografia seguro se seus textos cifrados revelassem 90% do texto simples, desde que 10% do texto simples permanecesse difícil de decifrar. Isso é claramente inaceitável na maioria das aplicações comuns de criptografia; por exemplo, ao criptografar um banco de dados de salários, ficaríamos justificadamente chateados se 90% dos salários dos funcionários fossem revelados!
- Deveria ser impossível para um invasor recuperar qualquer caractere do texto simples a partir do texto cifrado. Esta parece uma boa definição, mas ainda

não é suficiente. Voltando ao exemplo da criptografia de um banco de dados de salários, não consideraríamos um esquema de criptografia seguro se ele revelasse se o salário de um funcionário é maior ou menor que US\$ 100.000, mesmo que não revelasse nenhum dígito específico do salário desse funcionário. Da mesma forma, não gostaríamos que um esquema de criptografia revelasse se o funcionário A ganha mais do que o funcionário B. Outra questão é como formalizar o que significa para um adversário “recuperar um caractere do texto simples”. E se um invasor acertar, por pura sorte ou informação externa, que o dígito menos significativo do salário de alguém é 0? Claramente, isso não deve tornar um esquema de criptografia inseguro e, portanto, qualquer definição viável deve, de alguma forma, descartar tal comportamento como um ataque bem-sucedido.

- A resposta "certa": independentemente de qualquer informação que um invasor já possa, um texto cifrado não deve vaziar nenhuma informação adicional sobre o texto simples subjacente. Esta definição informal captura todas as preocupações descritas acima. Observe, em particular, que ela não tenta definir quais informações sobre o texto simples são "significativas"; ela simplesmente exige que nenhuma informação seja vazada. Isso é importante, pois significa que um esquema de criptografia seguro é adequado para todas as aplicações potenciais nas quais o sigilo é necessário.
- O que falta aqui é uma formulação matemática precisa da definição. Como devemos capturar o conhecimento prévio de um invasor sobre o texto simples? E o que significa (não) vaziar informações?

Agora que definimos uma meta de segurança, resta especificar um modelo de ameaça. Isso especifica qual "poder" se supõe que o invasor tenha, mas não impõe nenhuma restrição à estratégia do adversário. Esta é uma distinção importante: especificamos o que presumimos sobre as habilidades do adversário, mas não presumimos nada sobre como ele as utiliza. É impossível prever quais estratégias podem ser usadas em um ataque, e a história provou que tentativas de fazê-lo estão fadadas ao fracasso.

Existem várias opções plausíveis para o modelo de ameaça no contexto da criptografia; as opções padrão, em ordem crescente de poder do invasor, são:

- Ataque somente com texto cifrado: Este é o ataque mais básico e se refere a um cenário em que o adversário apenas observa um texto cifrado (ou múltiplos textos cifrados) e tenta determinar informações sobre o texto simples subjacente (ou textos simples).
- Ataque de texto simples conhecido: o adversário é capaz de aprender um ou mais pares de texto simples/texto cifrado gerados usando alguma chave. O objetivo do adversário é então deduzir informações sobre o texto simples subjacente de algum outro texto cifrado produzido usando a mesma chave. Todos os esquemas de criptografia clássicos que vimos são fáceis de quebrar usando um ataque de texto simples conhecido; deixamos uma demonstração como exercício.
- Ataque de texto simples escolhido: Neste ataque, o adversário pode obter pares de texto simples/texto cifrado (como acima) para textos simples de sua escolha.
- Ataque de texto cifrado escolhido: O último tipo de ataque é aquele em que o adversário também consegue obter (algumas informações sobre) a descryptografia de textos cifrados de sua escolha, por exemplo, se a

descriptografia de algum texto cifrado escolhido pelo invasor produz uma mensagem válida em inglês. O objetivo do adversário, mais uma vez, é obter informações sobre o texto simples subjacente de algum outro texto cifrado (cuja descriptografia o adversário não consegue obter diretamente). Nenhum desses modelos de ameaça é inerentemente melhor do que qualquer outro; o modelo correto a ser usado depende do ambiente em que o esquema de criptografia é implantado.

Os dois primeiros tipos de ataque são os mais fáceis de realizar. Em um ataque apenas com texto cifrado, a única coisa que o adversário precisa fazer é espionar o canal de comunicação público pelo qual as mensagens criptografadas são enviadas. Em um ataque com texto simples conhecido, presume-se que o adversário, de alguma forma, também obtenha textos cifrados correspondentes a textos simples conhecidos. Isso geralmente é fácil de realizar porque nem todas as mensagens criptografadas são confidenciais, pelo menos não indefinidamente. Como um exemplo trivial, duas partes podem sempre criptografar uma mensagem de "olá" sempre que começam a se comunicar. Como um exemplo mais complexo, a criptografia pode ser usada para manter os relatórios trimestrais de lucros em segredo até a data de sua divulgação; nesse caso, qualquer pessoa que espione o texto cifrado obterá posteriormente o texto simples correspondente.

Nos dois últimos ataques, presume-se que o adversário seja capaz de obter criptografias e/ou descriptografias de textos simples/cifrados de sua escolha.

Princípio 2 – Suposições Precisas: a maioria das construções criptográficas modernas não pode ser provada como segura incondicionalmente; tais provas exigiriam a resolução de questões na teoria da complexidade computacional que parecem longe de serem respondidas hoje. O resultado deste lamentável estado de coisas é que as provas de segurança normalmente se baseiam em suposições. A criptografia moderna exige que tais suposições sejam explícitas e matematicamente precisas. No nível mais básico, isso ocorre simplesmente porque as provas matemáticas de segurança exigem isso. Mas há outros motivos também:

1. Validação de suposições: Por sua própria natureza, suposições são afirmações que não são comprovadas, mas sim conjecturadas como verdadeiras. Para fortalecer nossa crença em alguma suposição, é necessário que a suposição seja estudada. Quanto mais a suposição for examinada e testada sem ser refutada, mais confiantes estaremos de que a suposição é verdadeira. Além disso, o estudo de uma suposição pode fornecer evidências de sua validade, mostrando que ela está implícita em alguma outra suposição que também é amplamente aceita.

Se a suposição em que se baseia não for declarada com precisão, ela não poderá ser estudada e (potencialmente) refutada. Assim, uma pré-condição para aumentar nossa confiança em uma suposição é ter uma declaração precisa do que exatamente está sendo assumido.

2. Comparação de esquemas: Frequentemente, em criptografia, somos apresentados a dois esquemas que podem ser comprovados como satisfazendo alguma definição, cada um baseado em uma suposição diferente. Assumindo que todo o resto é igual, qual esquema deve ser preferido? Se a suposição na qual o primeiro esquema se baseia for mais fraca do que a suposição na qual o segundo esquema se baseia (ou seja, a segunda suposição implica a primeira), então o primeiro esquema é preferível, pois pode acontecer que a segunda suposição seja falsa enquanto a primeira suposição seja verdadeira. Se as premissas utilizadas pelos dois esquemas não

forem comparáveis, a regra geral é preferir o esquema que se baseia na premissa mais bem estudada e na qual há maior confiança.

3. Compreendendo as premissas necessárias: Um esquema de criptografia pode ser baseado em algum bloco de construção subjacente. Se algumas fraquezas forem posteriormente encontradas no bloco de construção, como podemos dizer se o esquema de criptografia ainda é seguro? Se as premissas subjacentes referentes ao bloco de construção forem esclarecidas como parte da comprovação da segurança do esquema, então precisamos apenas verificar se as premissas necessárias são afetadas pelas novas fraquezas encontradas.

Uma questão que às vezes surge é: em vez de provar a segurança de um esquema com base em alguma outra premissa, por que não simplesmente assumir que a construção em si é segura? Em alguns casos — por exemplo, quando um esquema resistiu com sucesso a ataques por muitos anos — essa pode ser uma abordagem razoável. Mas essa abordagem nunca é preferível e é francamente perigosa quando um novo esquema está sendo introduzido. Os motivos acima ajudam a explicar o porquê. Primeiro, uma suposição que foi testada por vários anos é preferível a uma nova suposição ad hoc que é introduzida junto com uma nova construção. Segundo, há uma preferência geral por suposições que são mais simples de enunciar, uma vez que tais suposições são mais fáceis de estudar e (potencialmente) refutar. Assim, por exemplo, uma suposição de que algum problema matemático é difícil de resolver é mais simples de estudar e avaliar do que a suposição de que um esquema de criptografia satisfaz uma definição de segurança complexa. Outra vantagem de confiar em suposições de "nível inferior" (em vez de apenas assumir que uma construção é segura) é que essas suposições de baixo nível podem normalmente ser usadas em outras construções. Finalmente, suposições de baixo nível podem fornecer modularidade. Considere um esquema de criptografia cuja segurança depende de alguma propriedade presumida de um de seus blocos de construção. Se o bloco de construção subjacente não atender à suposição declarada, o esquema de criptografia ainda poderá ser instanciado usando um componente diferente que acredita-se que atenda aos requisitos necessários.

Princípio 3 – Provas de Segurança: os dois princípios descritos acima nos permitem atingir nosso objetivo de fornecer uma prova rigorosa de que uma construção satisfaz uma determinada definição sob certas suposições especificadas. Tais provas são especialmente importantes no contexto da criptografia, onde há um invasor tentando ativamente "quebrar" algum esquema. Provas de segurança fornecem uma garantia inabalável — em relação à definição e às suposições — de que nenhum invasor terá sucesso; isso é muito melhor do que adotar uma abordagem sem princípios ou heurística para o problema. Sem uma prova de que nenhum adversário com os recursos especificados pode quebrar algum esquema, nos resta apenas a nossa intuição de que esse é o caso. A experiência tem mostrado que a intuição em criptografia e segurança de computadores é desastrosa. Existem inúmeros exemplos de esquemas não comprovados que foram quebrados, às vezes imediatamente e às vezes anos após seu desenvolvimento.

C) Cifras de Bloco

Em uma cifra de bloco, a mensagem a ser criptografada é processada em blocos de k bits. Por exemplo, se $k = 64$, a mensagem é dividida em blocos de 64 bits e cada

bloco é criptografado independentemente. Para codificar um bloco, a cifra usa um mapeamento um-para-um para mapear o bloco de k bits de texto não criptografado para um bloco de k bits de texto cifrado. Vejamos um exemplo. Suponha que $k = 3$, de modo que a cifra de

cifrado). Um mapeamento possível é apresentado na Tabela 1. Observe que este é um mapeamento um-para-um; ou seja, há uma saída diferente para cada entrada. Essa cifra de bloco divide a mensagem em blocos de 3 bits e criptografa cada bloco de acordo com o mapeamento acima. Você deve verificar se a mensagem 010110001111 é criptografada em 101000111001.

input	output	input	output
000	110	100	011
001	111	101	010
010	101	110	000
011	100	111	001

Tabela 1

Observe que o mapeamento na Tabela 1 é apenas um dentre muitos mapeamentos possíveis. Quantos mapeamentos possíveis existem? Para responder a essa pergunta, observe que um mapeamento nada mais é do que uma permutação de todas as entradas possíveis. Existem 23 (= 8) entradas possíveis (listadas nas colunas de entrada). Essas oito entradas podem ser permutadas de $8! = 40.320$ maneiras diferentes. Como cada uma dessas permutações especifica um mapeamento, existem 40.320 mapeamentos possíveis. Podemos visualizar cada um desses mapeamentos como uma chave — se Alice e Bob conhecerem o mapeamento (a chave), eles podem criptografar e descriptografar as mensagens enviadas entre eles.

O ataque de força bruta para essa cifra consiste em tentar descriptografar o texto cifrado usando todos os mapeamentos. Com apenas 40.320 mapeamentos (quando $k = 3$), isso pode ser realizado rapidamente em um PC desktop. Para impedir ataques de força bruta, as cifras de bloco normalmente usam blocos muito maiores, consistindo de $k = 64$ bits ou até mais. Observe que o número de mapeamentos possíveis para uma cifra geral de k blocos é $2^k!$, o que é astronômico mesmo para valores moderados de k (como $k = 64$).

Embora as cifras de bloco de tabela completa, como acabamos de descrever, com valores moderados de k possam produzir esquemas robustos de criptografia de chave simétrica, infelizmente elas são difíceis de implementar. Para $k = 64$ e para um dado mapeamento, Alice e Bob precisariam manter uma tabela com 2^{64} valores de entrada, o que é uma tarefa inviável. Além disso, se Alice e Bob trocassem de chave, teriam que regenerar a tabela cada um.

As cifras de bloco normalmente usam funções que simulam tabelas permutadas aleatoriamente. Um exemplo (adaptado de [Kaufman2002]) de uma função como fork = 64 bits é mostrado na Figura 4. A função primeiro divide um bloco de 64 bits em 8 pedaços, com cada pedaço consistindo de 8 bits. Cada pedaço de 8 bits é processado por uma tabela de 8 bits para 8 bits, que é de tamanho gerenciável. Por exemplo, o

primeiro pedaço é processado pela tabela denotada por T1. Em seguida, os 8 blocos de saída são remontados em um bloco de 64 bits. As posições dos 64 bits no bloco são então embaralhadas (permutadas) para produzir uma saída de 64 bits. Essa saída é realimentada para a entrada de 64 bits, onde outro ciclo se inicia. Após n ciclos, a função fornece um bloco de texto cifrado de 64 bits. O objetivo das rodadas é fazer com que cada bit de entrada afete a maioria (se não todos) dos bits de saída finais. (Se apenas uma rodada fosse usada, um determinado bit de entrada afetaria apenas 8 dos 64 bits de saída.) A chave para esse algoritmo de cifra de bloco seriam as oito tabelas de permutação (assumindo que a função de embaralhamento seja de conhecimento público).

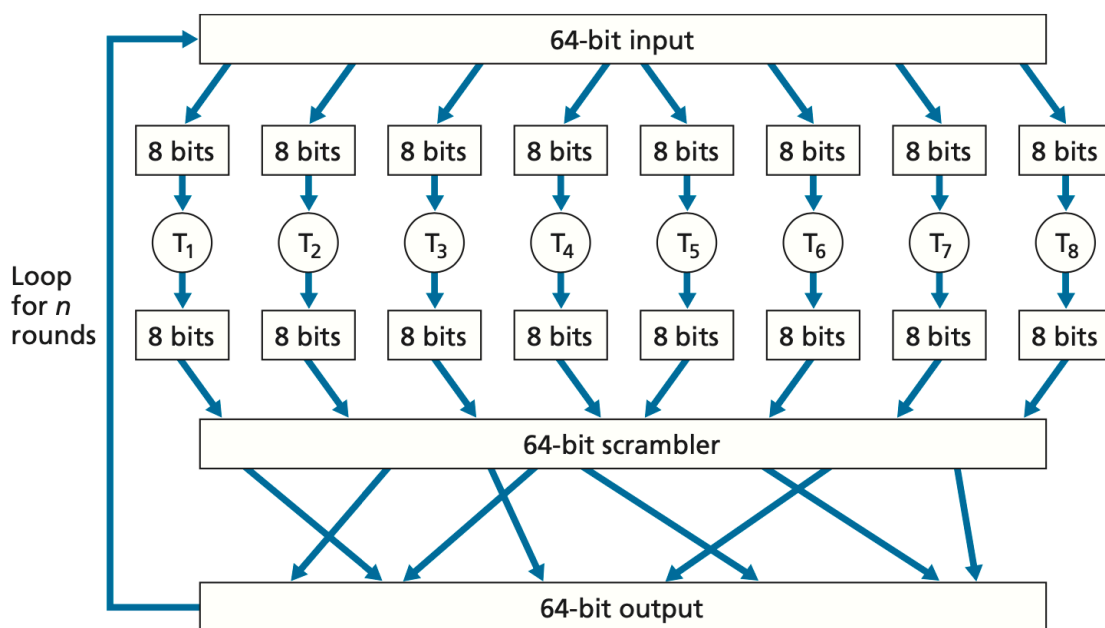


Figura 4

Atualmente, existem diversas cifras de bloco populares, incluindo DES (Data Encryption Standard), 3DES e AES (Advanced Encryption Standard). Cada um desses padrões utiliza funções, em vez de tabelas predeterminadas. Cada um desses algoritmos também utiliza uma sequência de bits como chave. Por exemplo, o DES utiliza blocos de 64 bits com uma chave de 56 bits. O AES utiliza blocos de 128 bits e pode operar com chaves de 128, 192 e 256 bits. A chave de um algoritmo determina os mapeamentos e permutações específicos da "minitabela" dentro dos componentes internos do algoritmo. O ataque de força bruta para cada uma dessas cifras consiste em percorrer todas as chaves, aplicando o algoritmo de descryptografia a cada chave. Observe que, com um comprimento de chave n , existem 2^n chaves possíveis. O NIST estima que uma máquina capaz de quebrar um DES de 56 bits em um segundo (ou seja, tentar todas as 256 chaves em um segundo) levaria aproximadamente 149 trilhões de anos para quebrar uma chave AES de 128 bits.

Encadeamento de Blocos de Cifra

A aplicar uma cifra de bloco conforme descrito anteriormente, simplesmente dividindo a mensagem em blocos de k bits e criptografando cada bloco independentemente,

ocorre um problema sutil, mas importante. Para verificar isso, observe que dois ou mais blocos de texto simples podem ser idênticos. Por exemplo, o texto simples em dois ou mais blocos poderia ser "HTTP/1.1". Para esses blocos idênticos, uma cifra de bloco produziria, obviamente, o mesmo texto cifrado. Um invasor poderia potencialmente adivinhar o texto simples ao ver blocos de texto cifrado idênticos e pode até mesmo ser capaz de descriptografar a mensagem inteira identificando blocos de texto cifrado idênticos e usando o conhecimento sobre a estrutura do protocolo subjacente.

Para resolver esse problema, pode-se adicionar alguma aleatoriedade ao texto cifrado para que blocos de texto simples idênticos produzam blocos de texto cifrado diferentes. Para explicar essa ideia, deixe $m(i)$ denotar o i -ésimo bloco de texto simples, $c(i)$ denotar o i -ésimo bloco de texto cifrado, e $a \oplus b$ denotar o exclusivo-ou (XOR) de duas cadeias de bits, a e b . (Lembre-se de que $0 \oplus 0 = 1 \oplus 1 = 0$ e $0 \oplus 1 = 1 \oplus 0 = 1$, e o XOR de duas cadeias de bits é feito em uma base bit a bit. Então, por exemplo, $10101010 \oplus 11110000 = 01011010$.) Também denota o algoritmo de criptografia de cifra de bloco com chave S como KS . A ideia básica é a seguinte. O remetente cria um número aleatório de k bits $r(i)$ para o i -ésimo bloco e calcula $c(i) = KS(m(i) \oplus r(i))$. Observe que um novo número aleatório de k bits é escolhido para cada bloco. O remetente então envia $c(1), r(1), c(2), r(2), c(3), r(3)$ e assim por diante. Como o destinatário recebe $c(i)$ e $r(i)$, ele pode recuperar cada bloco do texto simples calculando $m(i) = KS(c(i) \oplus r(i))$. É importante observar que, embora $r(i)$ seja enviado em texto simples e, portanto, possa ser detectado por Trudy, ela não pode obter o texto simples $m(i)$, pois não conhece a chave KS . Observe também que, se dois blocos de texto simples $m(i)$ e $m(j)$ forem iguais, os blocos de texto cifrado correspondentes $c(i)$ e $c(j)$ serão diferentes (desde que os números aleatórios $r(i)$ e $r(j)$ sejam diferentes, o que ocorre com altíssima probabilidade).

Como exemplo, considere a cifra de bloco de 3 bits na Tabela 1. Suponha que o texto simples seja 010010010. Se Alice criptografar isso diretamente, sem incluir a aleatoriedade, o texto cifrado resultante se tornará 101101101. Se Trudy detectar esse texto cifrado, como cada um dos três blocos de cifra é o mesmo, ela pode corretamente concluir que cada um dos três blocos de texto simples é o mesmo. Agora, suponha que Alice gere os blocos aleatórios $r(1) = 001$, $r(2) = 111$ e $r(3) = 100$ e use a técnica acima para gerar o texto cifrado $c(1) = 100$, $c(2) = 010$ e $c(3) = 000$. Observe que os três blocos de texto cifrado são diferentes, embora os blocos de texto simples sejam os mesmos. Alice então envia $c(1), r(1), c(2)$ e $r(2)$. Você deve verificar se Bob consegue obter o texto simples original usando a chave compartilhada KS .

A introdução da aleatoriedade resolve um problema, mas cria outro: a saber, Alice precisa transmitir o dobro de bits do que antes. De fato, para cada bit da cifra, ela agora precisa enviar um bit aleatório, dobrando a largura de banda necessária. As cifras de bloco normalmente usam uma técnica chamada Encadeamento de Blocos de Cifras (CBC). A ideia básica é enviar apenas um valor aleatório junto com a primeira mensagem e, em seguida, fazer com que o remetente e o destinatário usem os blocos codificados computados no lugar do número aleatório subsequente. Especificamente, o CBC opera da seguinte forma:

1. Antes de criptografar a mensagem (ou o fluxo de dados), o remetente gera uma sequência aleatória de k bits, chamada Vetor de Inicialização (VI). Denote esse vetor de inicialização por $c(0)$. O remetente envia o VI ao destinatário em texto não criptografado.

2. Para o primeiro bloco, o remetente calcula $m(1) \oplus c(0)$, ou seja, calcula o valor exclusivo ou do primeiro bloco de texto simples com o IV. Em seguida, ele executa o resultado através do algoritmo de cifra de bloco para obter o bloco de texto cifrado correspondente; ou seja,
 $c(1) = KS(m(1) \oplus c(0))$. O remetente envia o bloco criptografado $c(1)$ para o destinatário.
3. Para o i -ésimo bloco, o remetente gera o i -ésimo bloco de texto cifrado a partir de $c(i) = KS(m(i) \oplus c(i - 1))$.

Vamos agora examinar algumas das consequências dessa abordagem. Primeiro, o receptor ainda será capaz de recuperar a mensagem original. De fato, quando o receptor recebe $c(i)$, ele a decifra com KS para obter $s(i) = m(i) \oplus c(i - 1)$; como o receptor também conhece $c(i - 1)$, ele então obtém o bloco de texto simples de $m(i) = s(i) \oplus c(i - 1)$. Segundo, mesmo que dois blocos de texto simples sejam idênticos, os textos cifrados correspondentes (quase sempre) serão diferentes. Terceiro, embora o remetente envie o IV em texto simples, um intruso ainda não será capaz de decifrar os blocos de texto cifrado, visto que o intruso não conhece a chave secreta, S. Por fim, o remetente envia apenas um bloco de overhead (o IV), aumentando assim de forma insignificante o uso de largura de banda para mensagens longas (compostas por centenas de blocos).

Como exemplo, vamos agora determinar o texto cifrado para a cifra de bloco de 3 bits na Tabela 1 com o texto simples 010010010 e $IV = c(0) = 001$. O remetente primeiro usa o IV para calcular $c(1) = KS(m(1) \oplus c(0)) = 100$. O remetente então calcula $c(2) = KS(m(2) \oplus c(1)) = KS(010 \oplus 100) = 000$, e $c(3) = KS(m(3) \oplus c(2)) = KS(010 \oplus 000) = 101$. O leitor deve verificar se o destinatário, conhecendo o IV e o KS, consegue recuperar o texto simples original.

O CBC tem uma consequência importante ao projetar protocolos de rede seguros: precisaremos fornecer um mecanismo dentro do protocolo para distribuir o IV do remetente para o destinatário.

D) Criptografia de Chave Publica

Por mais de 2.000 anos (desde a época da cifra de César até a década de 1970), a comunicação criptografada exigia que as duas partes comunicantes compartilhassem um segredo comum — a chave simétrica usada para cifrar e decifrar. Uma dificuldade com essa abordagem é que as duas partes precisam, de alguma forma, concordar com a chave compartilhada; mas isso, por si só, requer uma comunicação segura. Talvez as partes pudessem primeiro se encontrar e concordar com a chave pessoalmente e, posteriormente, se comunicar por meio de criptografia. Em um mundo em rede, no entanto, as partes comunicantes podem nunca se encontrar e nunca conversar, exceto pela rede.

É possível que duas partes se comuniquem por meio de criptografia sem ter uma chave secreta compartilhada que seja conhecida antecipadamente? Em 1976, Diffie e Hellman demonstraram um algoritmo (conhecido hoje como Troca de Chaves Diffie-Hellman) para fazer exatamente isso — uma abordagem radicalmente diferente e maravilhosamente elegante para a comunicação segura, que levou ao desenvolvimento dos atuais sistemas de criptografia de chave pública.

O uso da criptografia de chave pública é conceitualmente bastante simples. Suponha que Alice queira se comunicar com Bob. Como mostrado na Figura 5, em vez de Bob e Alice compartilharem uma única chave secreta (como no caso de sistemas de chave simétrica), Bob (o destinatário das mensagens de Alice) possui duas chaves — uma chave pública disponível para todos no mundo (incluindo Trudy, a intrusa) e uma chave privada conhecida apenas por Bob. Usaremos a notação K_B^+ e K_B^- para nos referirmos às chaves pública e privada de Bob, respectivamente. Para se comunicar com Bob, Alice primeiro busca a chave pública de Bob. Alice então criptografa sua mensagem, m , para Bob usando a chave pública de Bob e um algoritmo de criptografia conhecido (por exemplo, padronizado); ou seja, Alice calcula $K_B^+(m)$. Bob recebe a mensagem criptografada de Alice e usa sua chave privada e um algoritmo de descryptografia conhecido (por exemplo, padronizado) para descryptografar a mensagem criptografada de Alice. Ou seja, Bob calcula $K_B^-(K_B^+(m))$. Existem algoritmos e técnicas de criptografia/descryptografia para escolher chaves públicas e privadas tais que $K_B^-(K_B^+(m)) = m$; ou seja, aplicar a chave pública de Bob, K_B^+ , a uma mensagem, m (para obter $K_B^+(m)$) e, em seguida, aplicar a chave privada de Bob, K_B^- , à versão criptografada de m (ou seja, calcular $K_B^-(K_B^+(m))$) retorna m . Dessa forma, Alice pode usar a chave pública de Bob para enviar uma mensagem secreta a Bob sem que nenhum dos dois precise distribuir nenhuma chave secreta. Veremos em breve que podemos trocar a criptografia da chave pública e da chave privada e obter o mesmo resultado — ou seja, $K_B^-(K_B^+(m)) = K_B^+(K_B^-(m)) = m$.

Embora a criptografia de chave pública seja atraente, uma preocupação imediatamente vem à mente. Como a chave de criptografia de Bob é pública, qualquer pessoa pode enviar uma mensagem criptografada para ele, incluindo Alice ou alguém que finja ser Alice. No caso de uma única chave secreta compartilhada, o fato de o remetente conhecer a chave secreta identifica implicitamente o remetente para o destinatário. No caso da criptografia de chave pública, no entanto, isso não é mais o caso, pois qualquer pessoa pode enviar uma mensagem criptografada para Bob usando a chave pública de Bob.

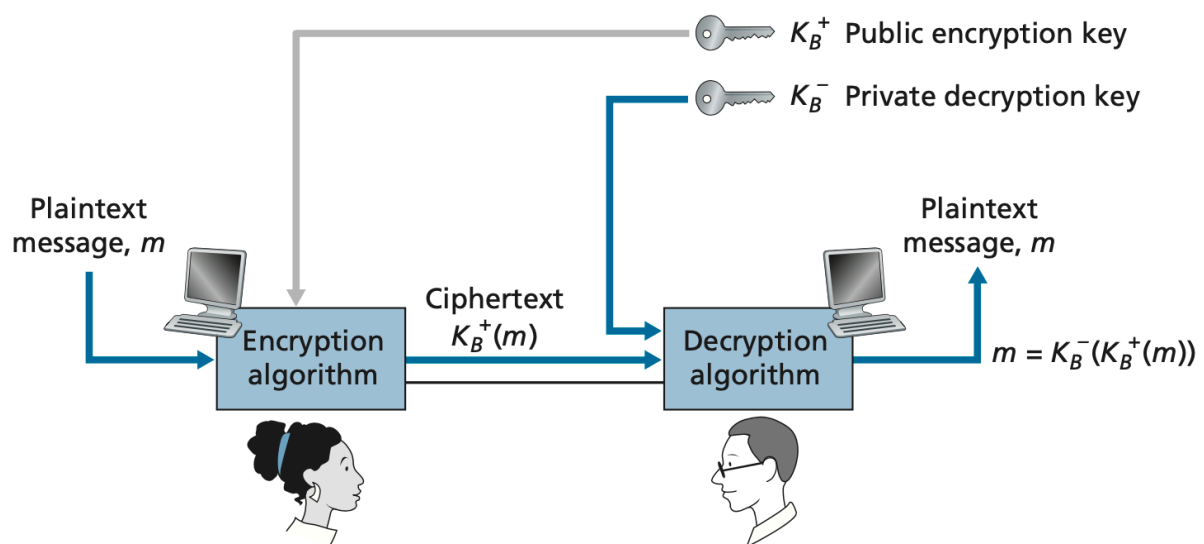


Figura 5

RSA

O algoritmo RSA (nomeado em homenagem aos seus fundadores, Ron Rivest, Adi Shamir e Leonard Adleman) tornou-se quase sinônimo de criptografia de chave pública.

O RSA faz uso extensivo de operações aritméticas usando aritmética módulo- n . Portanto, vamos revisar brevemente a aritmética modular. Lembre-se de que $x \bmod n$ significa simplesmente o resto de x quando dividido por n ; então, por exemplo, $19 \bmod 5 = 4$. Na aritmética modular, realizam-se as operações usuais de adição, multiplicação e exponenciação. No entanto, o resultado de cada operação é substituído pelo resto inteiro que resta quando o resultado é dividido por n . A adição e a multiplicação com aritmética modular são facilitadas pelos seguintes fatos úteis:

$$[(a \bmod n) + (b \bmod n)] \bmod n = (a + b) \bmod n$$

$$[(a \bmod n) - (b \bmod n)] \bmod n = (a - b) \bmod n$$

$$[(a \bmod n) \cdot (b \bmod n)] \bmod n = (a \cdot b) \bmod n$$

- $(a \bmod n)^d \bmod n = a^d \bmod n$ é uma identidade

Agora, suponha que Alice queira enviar a Bob uma mensagem criptografada em RSA, como mostrado na Figura 5. Uma mensagem nada mais é do que um padrão de bits, e cada padrão de bits pode ser representado exclusivamente por um número inteiro (junto com o comprimento do padrão de bits). Por exemplo, suponha que uma mensagem seja o padrão de bits 1001; Esta mensagem pode ser representada pelo número inteiro decimal 9. Portanto, criptografar uma mensagem com RSA equivale a criptografar o número inteiro único que representa a mensagem.

Existem dois componentes inter-relacionados do RSA:

- A escolha da chave pública e da chave privada
- O algoritmo de criptografia e descriptografia

Para gerar as chaves RSA pública e privada, Bob executa as seguintes etapas:

1. Escolha dois números primos grandes, p e q . Qual deve ser o tamanho de p e q ? Quanto maiores os valores, mais difícil é quebrar o RSA, mas mais tempo leva para realizar a codificação e a decodificação. Os Laboratórios RSA recomendam que o produto de p e q seja da ordem de 1.024 bits.
2. Computar $n = pq$ e $z = (p - 1)(q - 1)$.
3. Escolha um número, e , menor que n , que não tenha fatores comuns (além de 1) com z . (Neste caso, diz-se que e e z são primos relativos.) A letra e é usada pois este valor será usado na criptografia.
4. Encontre um número, d , tal que $ed - 1$ seja exatamente divisível (ou seja, sem resto) por z . A letra d é usada porque este valor será usado na descriptografia. Em outras palavras, dado e , escolhemos d tal que $ed \bmod z = 1$.
5. A chave pública que Bob disponibiliza ao mundo, KB^+ , é o par de números (n, e) ; sua chave privada, KB^- , é o par de números (n, d) .

A criptografia por Alice e a descriptografia por Bob são feitas da seguinte forma:

- Suponha que Alice queira enviar a Bob um padrão de bits representado pelo número inteiro m (com $m < n$). Para codificar, Alice realiza a exponenciação m^e e, em seguida, calcula o resto inteiro quando m^e é dividido por n . Em outras palavras, o valor criptografado, c , da mensagem de texto simples de Alice, m , é $c = m^e \bmod n$. O padrão de bits correspondente a este texto cifrado c é enviado a Bob.
- Para descryptografar a mensagem de texto cifrado recebida, c , Bob calcula $m = c^d \bmod n$ o que requer o uso de sua chave privada (n, d) .

Como um exemplo simples de RSA, suponha que Bob escolha $p = 5$ e $q = 7$. (É verdade que esses valores são muito pequenos para serem seguros.) Então, $n = 35$ e $z = 24$. Bob escolhe $e = 5$, já que 5 e 24 não têm fatores comuns. Finalmente, Bob escolhe $d = 29$, já que $5 \cdot 29 - 1$ (ou seja, $ed - 1$) é exatamente divisível por 24. Bob torna os dois valores, $n = 35$ e $e = 5$, públicos e mantém o valor $d = 29$ em segredo. Observando esses dois valores públicos, suponha que Alice agora queira enviar as letras l, o, v e e para Bob. Interpretando cada letra como um número entre 1 e 26 (sendo a 1 e z 26), Alice e Bob realizam a criptografia e a descryptografia mostradas nas Tabelas 2 e 3, respectivamente. Observe que, neste exemplo, consideramos cada uma das quatro letras como uma mensagem distinta. Um exemplo mais realista seria converter as quatro letras em suas representações ASCII de 8 bits e, em seguida, criptografar o inteiro correspondente ao padrão de 32 bits resultante.

Plaintext Letter	m : numeric representation	m^e	Ciphertext $c = m^e \bmod n$
l	12	248832	17
o	15	759375	15
v	22	5153632	22
e	5	3125	10

Tabela 2

Ciphertext c	c^d	$m = c^d \bmod n$	Plaintext Letter
17	4819685721067509150915091411825223071697	12	l
15	127834039403948858939111232757568359375	15	o
22	851643319086537701956194499721106030592	22	v
10	10000000000000000000000000000000	5	e

Tabela 3

Dado que o exemplo nas Tabelas 2 e 3 já produziu alguns números extremamente grandes, e visto que vimos anteriormente que p e q devem ter centenas de bits cada, várias questões práticas relacionadas ao RSA vêm à mente. Como escolher números primos grandes? Como escolher e e d ? Como realizar a exponenciação com números grandes?

Chaves de Sessão

A exponenciação exigida pelo RSA é um processo bastante demorado. Como resultado, o RSA é frequentemente usado na prática em combinação com a criptografia de chave simétrica. Por exemplo, se Alice quiser enviar a Bob uma grande quantidade de dados criptografados, ela poderia fazer o seguinte. Primeiro, Alice escolhe uma chave que será usada para codificar os próprios dados; essa chave é chamada de chave de sessão e é denotada por KS . Alice deve informar Bob sobre a chave de sessão, pois esta é a chave simétrica compartilhada que eles usarão com uma cifra de chave simétrica (por exemplo, com DES ou AES). Alice criptografa a chave de sessão usando a chave pública de Bob, ou seja, calcula $c = (KS)^e \bmod n$. Bob recebe a chave de sessão criptografada em RSA, c , e a descriptografa para obter a chave de sessão, KS . Bob agora sabe a chave de sessão que Alice usará para sua transferência de dados criptografados.

E) Código de autenticação de mensagens

Ao trabalhar o problema de integridade de mensagens, com a compreensão das funções de hash, vamos revisar como pode ser avaliada a integridade das mensagens:

1. Alice cria a mensagem m e calcula o hash $H(m)$
2. Alice então anexa $H(m)$ à mensagem m , criando uma mensagem estendida $(m, H(m))$, e a envia para Bob.
3. Bob recebe uma mensagem estendida (m, h) e calcula $H(m)$. Se $H(m) = h$, Bob conclui que está tudo bem.

Essa abordagem é obviamente falha : Trudy pode criar uma mensagem falsa m' na qual ela diz ser Alice, calcular $H(m')$ e enviar $(m', H(m'))$ para Bob. Quando Bob recebe a mensagem, tudo se encaixa na etapa 3, então Bob não suspeita de nada.

Para verificar a integridade da mensagem, além de usar funções de hash criptográficas, Alice e Bob precisarão de um segredo compartilhado. Esse segredo compartilhado, que nada mais é do que uma sequência de bits, é chamado de chave de autenticação.

Usando esse segredo compartilhado, a integridade da mensagem pode ser realizada da seguinte forma:

1. Alice cria a mensagem m , concatena s com m para criar $m + s$ e calcula o hash $H(m + s)$. $H(m + s)$ é chamado de código de autenticação de mensagem (MAC – *Message Authentication Code*).
2. Alice então anexa o MAC à mensagem m , criando uma mensagem estendida $(m, H(m + s))$, e a envia para Bob.
3. Bob recebe uma mensagem estendida (m, h) e, sabendo s , calcula o MAC $H(m + s)$. Se $H(m + s) = h$, Bob conclui que está tudo bem.

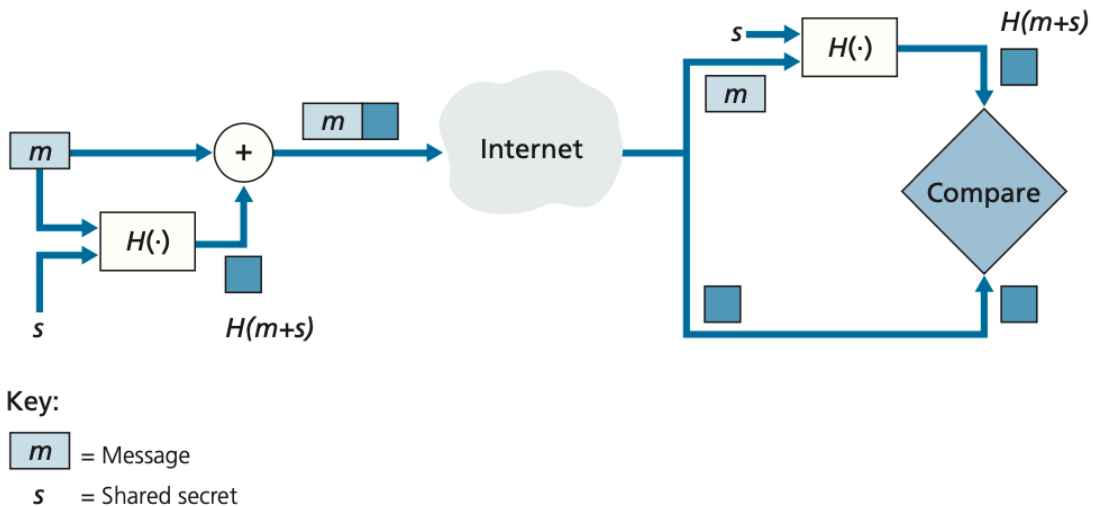


Figura 6. Geração do MAC

Usando um MAC, as entidades podem autenticar as mensagens que enviam umas às outras sem precisar integrar algoritmos complexos de criptografia no processo de integridade.

Existem diversos padrões para MACs que foram propostos ao longo dos anos. O padrão mais popular atualmente é o HMAC, que pode ser usado com MD5 ou SHA-1. O HMAC, na verdade, executa os dados e a chave de autenticação por meio da função hash duas vezes. Resta ainda uma questão importante: como distribuímos a chave de autenticação compartilhada para as entidades que se comunicam?

F) Assinaturas Digitais

Em um mundo digital, muitas vezes queremos indicar o proprietário ou criador de um documento, ou demonstrar nossa concordância com o conteúdo de um documento. Uma assinatura digital é uma técnica criptográfica para atingir esses objetivos em um mundo digital.

Assim como acontece com as assinaturas manuscritas, a assinatura digital deve ser feita de forma verificável e infalsificável. Ou seja, deve ser possível provar que um documento assinado por um indivíduo foi de fato assinado por esse indivíduo (a assinatura deve ser verificável) e que somente esse indivíduo poderia ter assinado o documento (a assinatura não pode ser falsificada).

Vamos agora considerar como podemos projetar um esquema de assinatura digital. Observe que, quando Bob assina uma mensagem, ele deve inserir algo na mensagem que seja exclusivo para ele. Bob poderia considerar anexar um MAC para a assinatura, onde o MAC é criado anexando sua chave (exclusiva para ele) à mensagem e, em seguida, obtendo o hash. Mas, para Alice verificar a assinatura, ela também precisa ter uma cópia da chave; nesse caso, a chave não seria exclusiva para Bob. Portanto, os MACs não funcionarão aqui.

Lembre-se de que, com a criptografia de chave pública, Bob possui uma chave pública e uma privada, sendo ambas exclusivas para Bob. Portanto, a criptografia de chave pública é uma excelente candidata para fornecer assinaturas digitais.

Suponha que Bob queira assinar digitalmente um documento, m . Podemos pensar no documento como um arquivo ou uma mensagem que Bob irá assinar e enviar. Como mostrado na Figura 7, para assinar este documento, Bob simplesmente usa sua chave privada, K_B^- , para calcular $K_B^-(m)$. À primeira vista, pode parecer estranho que Bob esteja usando sua chave privada para assinar um documento. Mas lembre-se de que criptografia e descryptografia nada mais são do que operações matemáticas e lembre-se de que o objetivo de Bob não é embaralhar ou obscurecer o conteúdo do documento, mas sim assiná-lo de uma maneira que seja verificável e infalsificável. A assinatura digital do documento de Bob é $K_B^-(m)$.

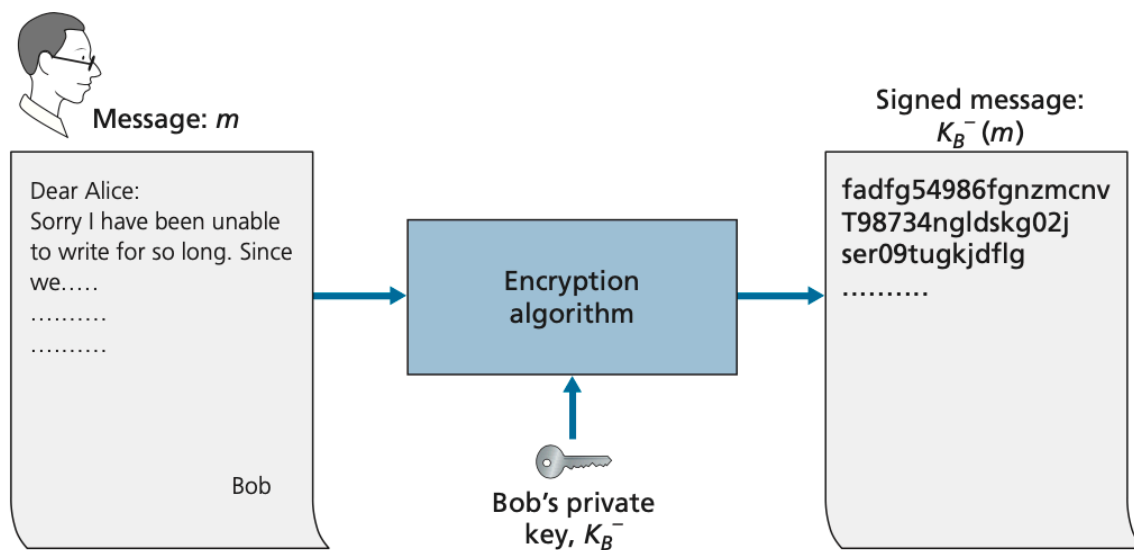


Figura 7. Processo de Assinatura Digital

A assinatura digital $K_B^-(m)$ atende aos nossos requisitos de ser verificável e infalsificável? Suponha que Alice tenha m e $K_B^-(m)$. Ela quer provar em tribunal que Bob de fato assinou o documento e era a única pessoa que poderia tê-lo assinado. Alice pega a chave pública de Bob, K_B^+ , e a aplica à assinatura digital, $K_B^-(m)$, associada ao documento, m . Ou seja, ela calcula $K_B^+(K_B^-(m))$ e produz m , que corresponde exatamente ao documento original. Alice então argumenta que somente Bob poderia ter assinado o documento, pelos seguintes motivos:

- Quem assinou a mensagem deve ter usado a chave privada, K_B^- , ao calcular a assinatura $K_B^-(m)$, tal que $K_B^+(K_B^-(m)) = m$.
- A única pessoa que poderia saber a chave privada, K_B^- , é Bob, dado que com o RSA o fato de conhecer a chave pública, K_B^+ , não permite conhecer a chave privada, K_B^- . Portanto, a única pessoa que poderia conhecer K_B^- é a pessoa que gerou o par de chaves, (K_B^+, K_B^-) , em primeiro lugar, Bob (Observe que isso pressupõe, no entanto, que Bob não deu K_B^- a ninguém, nem ninguém roubou K_B^- de Bob.).

Também é importante observar que, se o documento original, m , for modificado para alguma forma alternativa, m' , a assinatura que Bob criou para m não será válida para m' ,

já que $K_B + (K_B - (m))$ não é igual a m' . Assim, vemos que as assinaturas digitais também fornecem integridade à mensagem, permitindo ao destinatário verificar se a mensagem não foi alterada, bem como a origem da mensagem.

Uma preocupação com a assinatura de dados por criptografia é que a criptografia e a descryptografia são computacionalmente caras. Uma abordagem mais eficiente é introduzir funções de hash na assinatura digital.

Um algoritmo de hash recebe uma mensagem, m , de comprimento arbitrário e calcula uma "impressão digital" de comprimento fixo da mensagem, denotada por $H(m)$. Usando uma função de hash, Bob assina o hash de uma mensagem em vez da mensagem em si, ou seja, Bob calcula $K_B - (H(m))$. Como $H(m)$ é geralmente muito menor que a mensagem original m , o esforço computacional necessário para criar a assinatura digital é substancialmente reduzido.

No contexto de Bob enviando uma mensagem para Alice, a Figura 8 fornece um resumo do procedimento operacional para a criação de uma assinatura digital. Bob submete sua longa mensagem original a uma função de hash. Ele então assina digitalmente o hash resultante com sua chave privada. A mensagem original (em texto simples), juntamente com o resumo da mensagem assinado digitalmente (doravante denominado assinatura digital), é então enviada para Alice. A Figura 9 fornece um resumo do procedimento operacional da assinatura. Alice aplica a chave pública do remetente à mensagem para obter um resultado de hash. Alice também aplica a função hash à mensagem em texto simples para obter um segundo resultado hash. Se os dois hashes corresponderem, Alice poderá ter certeza sobre a integridade e o autor da mensagem.

A comparação do processo de assinatura digital e MACs têm paralelos, mas também apresentam diferenças importantes. Tanto as assinaturas digitais quanto os MACs começam com uma mensagem (ou um documento). Para criar um MAC a partir da mensagem, anexamos uma chave de autenticação à mensagem e, em seguida, obtemos o hash do resultado. Observe que nem a chave pública nem a criptografia de chave simétrica estão envolvidas na criação do MAC. Para criar uma assinatura digital, primeiro obtemos o hash da mensagem e, em seguida, criptografamos a mensagem com nossa chave privada (usando criptografia de chave pública). Portanto, uma assinatura digital é uma técnica "mais pesada", pois requer uma Infraestrutura de Chave Pública (ICP) subjacente com autoridades de certificação.

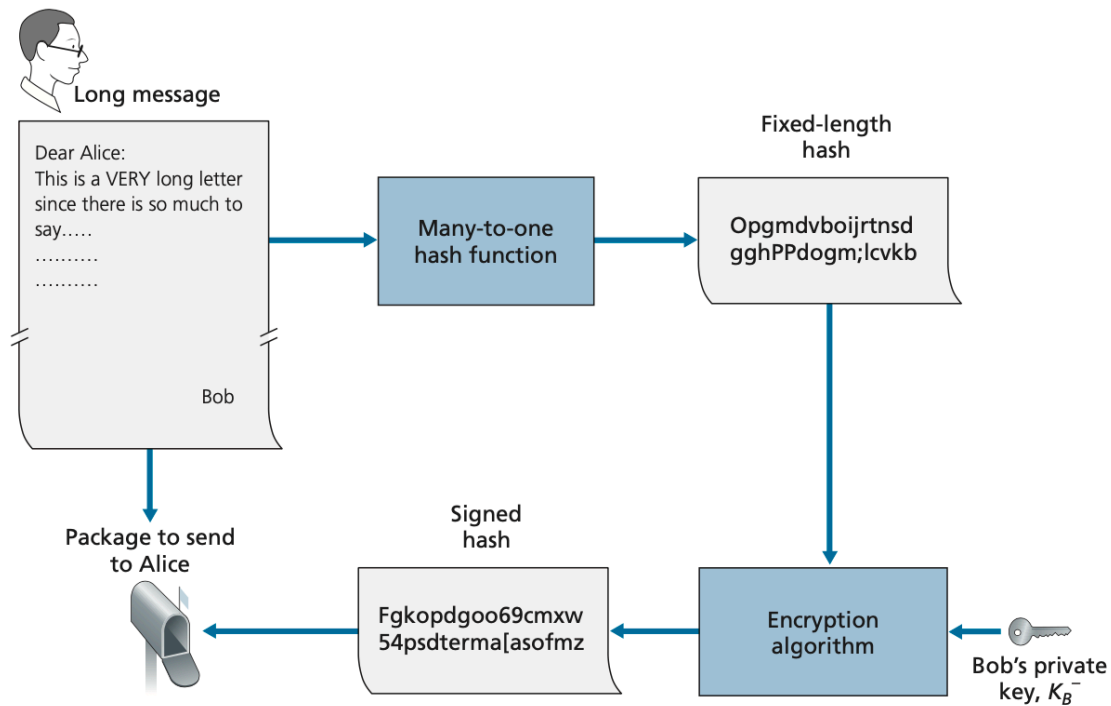


Figura 8. Criação de uma assinatura digital

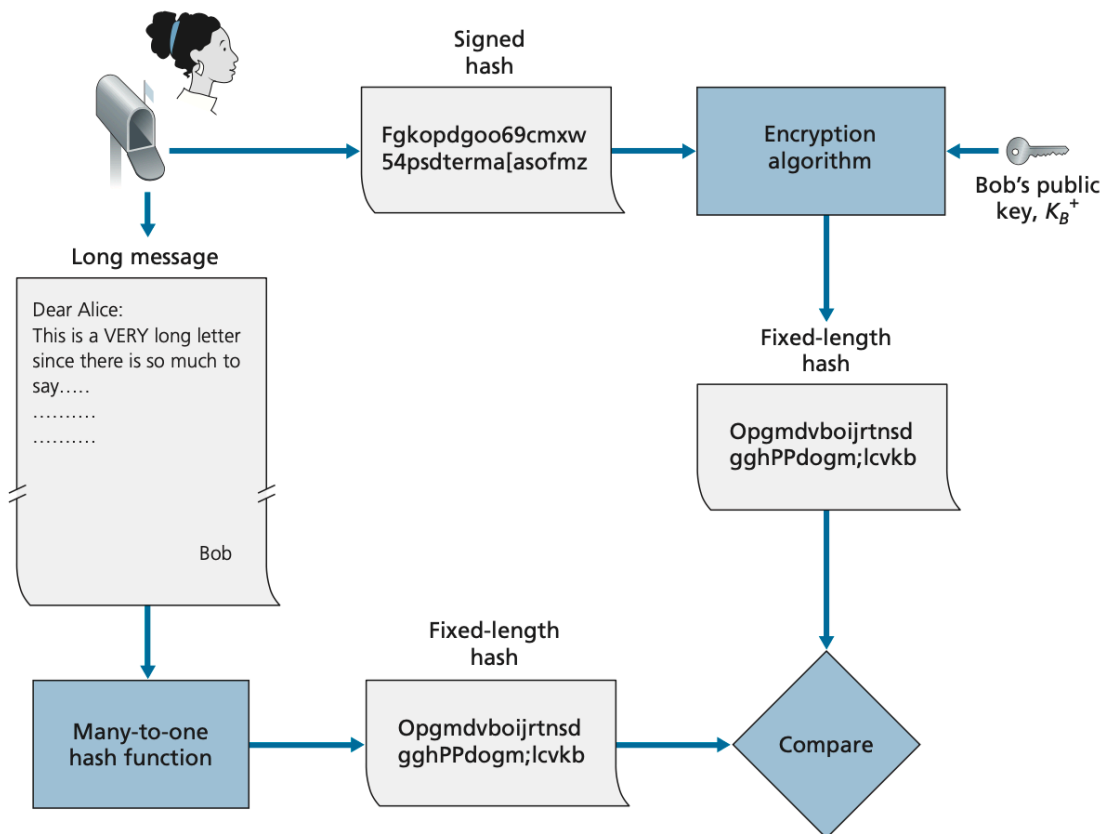


Figura 9. Verificação de uma assinatura Digital

G) Certificação de Chave Pública

Uma aplicação importante das assinaturas digitais é a certificação de chave pública, ou seja, a certificação de que uma chave pública pertence a uma entidade específica. A certificação de chave pública é usada em muitos protocolos populares de rede segura, incluindo IPsec e TLS.

Para entender esse problema, vamos considerar a seguinte situação: Alice trabalha com entrega de pizza e aceita pedidos pela internet. Bob envia a Alice uma mensagem de texto simples que inclui seu endereço residencial e o tipo de pizza que deseja. Nessa mensagem, Bob também inclui uma assinatura digital (ou seja, um hash assinado da mensagem de texto simples original) para provar a Alice que ele é a verdadeira fonte da mensagem. Para verificar a assinatura, Alice obtém a chave pública de Bob (talvez de um servidor de chaves públicas ou da mensagem de e-mail) e verifica a assinatura digital. Dessa forma, ela garante que Bob fez o pedido. Mas, como mostrado na Figura 10, Trudy envia uma mensagem para Alice na qual diz ser Bob, informa o endereço residencial de Bob e pede uma pizza. Nessa mensagem, ela também inclui sua chave pública (a de Trudy), embora Alice naturalmente presume que seja a chave pública de Bob. Trudy também anexa uma assinatura digital, que foi criada com sua própria chave privada (a de Trudy). Após receber a mensagem, Alice aplica a chave pública de Trudy (pensando que seja a de Bob) à assinatura digital e conclui que a mensagem em texto simples foi de fato criada por Bob.

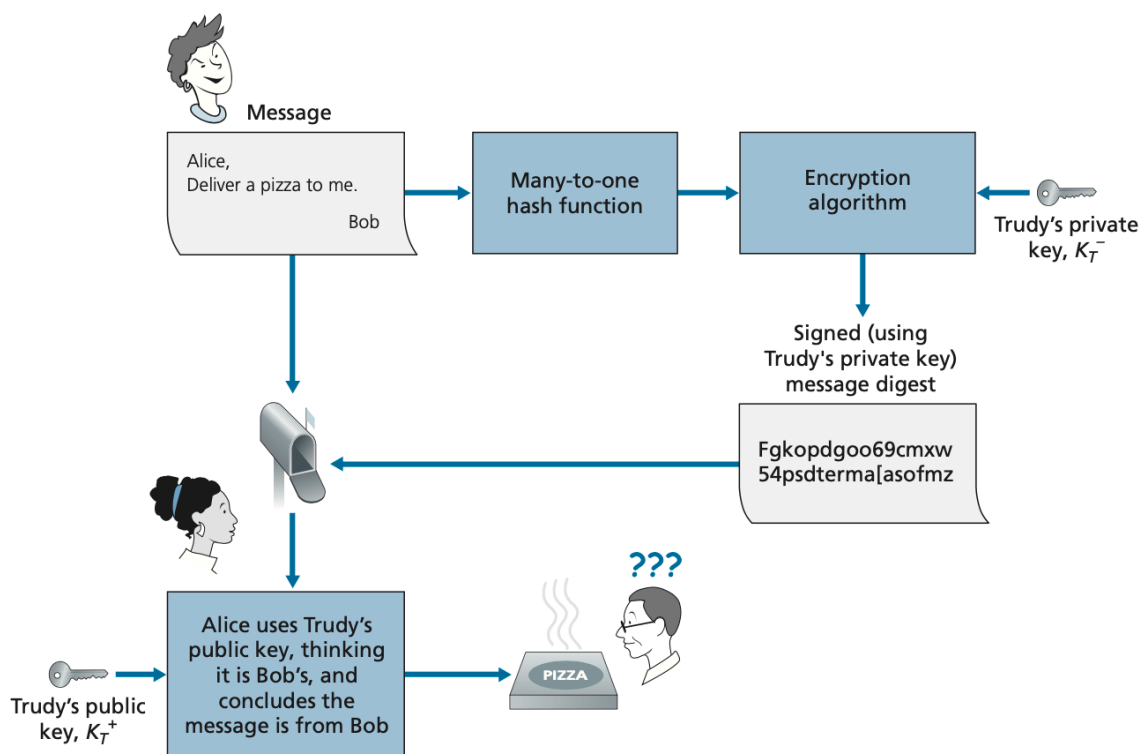


Figura 10. Trudy se fazendo passar por Bob

Concluimos que para que a criptografia de chave pública seja útil, é necessário verificar que a chave pública é da entidade com a qual deseja se comunicar. Por exemplo, quando Alice deseja se comunicar com Bob usando criptografia de chave

pública, ela precisa verificar se a chave pública que supostamente pertence a Bob é de fato a dele.

A vinculação de uma chave pública a uma entidade específica é normalmente realizada por uma Autoridade Certificadora (AC), cuja função é validar identidades e emitir certificados. Uma AC tem as seguintes funções:

1. Uma AC verifica se uma entidade (uma pessoa, um roteador e assim por diante) é quem diz ser. Não há procedimentos obrigatórios para a certificação. Ao lidar com uma AC, é preciso confiar que ela realizou uma verificação de identidade rigorosa e adequada. Normalmente tende a dar mais confiança uma AC que faça parte de um programa federal ou estadual. Só se pode confiar na identidade associada a uma chave pública na medida em que se pode confiar em uma AC e em suas técnicas de verificação de identidade.
2. Depois que a AC verifica a identidade da entidade, ela cria um certificado que vincula a chave pública da entidade à identidade. O certificado contém a chave pública e informações de identificação globalmente exclusivas sobre o proprietário da chave pública (por exemplo, um nome humano ou um endereço IP). O certificado é assinado digitalmente pela AC. Essas etapas são mostradas na Figura 11.

Com a entrada do AC no processo, quando Bob faz seu pedido, ele também envia seu certificado assinado pela AC. Alice usa a chave pública da AC para verificar a validade do certificado de Bob e extrair a chave pública de Bob.

Tanto a União Internacional de Telecomunicações (UIT) quanto a IETF desenvolveram padrões para ACs. O ITU X.509 [especifica um serviço de autenticação, bem como uma sintaxe específica para certificados. O RFC 1422 descreve o gerenciamento de chaves baseado em AC para uso com e-mail seguro da Internet. É compatível com o X.509, mas vai além do X.509 ao estabelecer procedimentos e convenções para uma arquitetura de gerenciamento de chaves.

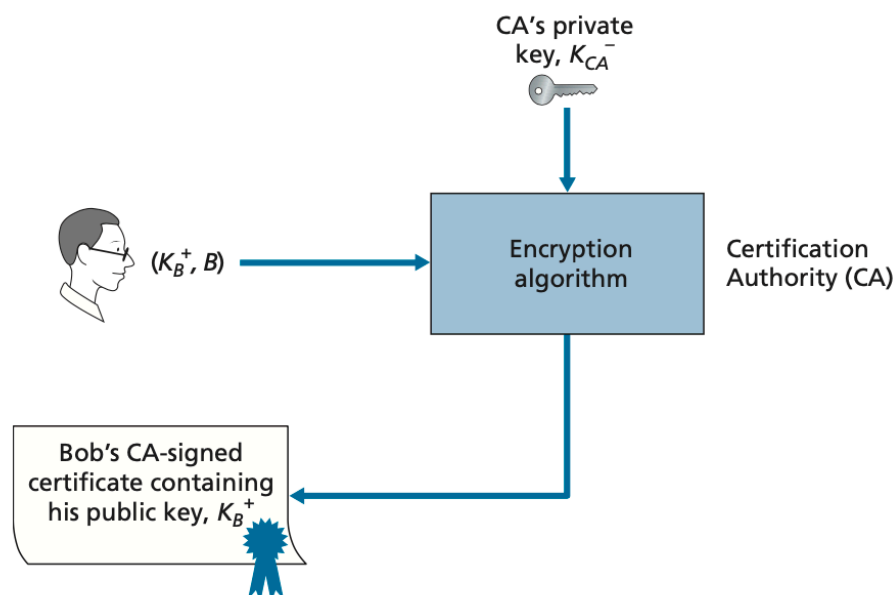


Figura 11. Certificação da Chave Pública de Bob

Exercícios

1. Utilizando a cifra monoalfabética da Figura 2, codifique a mensagem “Este é um problema fácil”. Decodifique a mensagem “rmij’u uamu xyj”.
2. Considere o sistema polialfabético mostrado na Figura 3. Um ataque ao texto simples escolhido, capaz de obter a codificação em texto simples da mensagem “A rápida raposa marrom salta sobre o cão preguiçoso”, será suficiente para decodificar todas as mensagens? Por que sim ou por que não?
3. Considere a cifra de bloco na Figura 4. Suponha que cada cifra de bloco T_i simplesmente inverta a ordem dos oito bits de entrada (de modo que, por exemplo, 11110000 se torne 00001111). Suponha ainda que o embaralhador de 64 bits não modifique nenhum bit (de modo que o valor de saída do m -ésimo bit seja igual ao valor de entrada do m -ésimo bit).
 - a. Com $n = 3$ e a entrada original de 64 bits igual a 10100000 repetida oito vezes, qual é o valor da saída?
 - b. Repita a parte (a), mas agora altere o último bit da entrada original de 64 bits de 0 para 1.
 - c. Repita as partes (a) e (b), mas agora suponha que o embaralhador de 64 bits inverta a ordem dos 64 bits.
4. Considere a cifra de bloco de 3 bits na Tabela 1. Suponha que o texto simples seja 100100100. (a) Inicialmente, suponha que CBC não seja usado. Qual é o texto cifrado resultante? (b) Suponha que Trudy detecte o texto cifrado. Supondo que ela saiba que uma cifra de bloco de 3 bits sem CBC está sendo empregada (mas não conhece a cifra específica), o que ela pode supor? (c) Agora, suponha que CBC seja usado com $IV = 111$. Qual é o texto cifrado resultante?
5. (a) Usando RSA, escolha $p = 3$ e $q = 11$ e codifique a palavra “cachorro” criptografando cada letra separadamente. Aplique o algoritmo de descryptografia à versão criptografada para recuperar a mensagem original em texto simples.
 - a. (b) Repita o item (a), mas agora criptografe “cachorro” como uma única mensagem m .
6. Considere uma variação do algoritmo MAC em que o remetente envia $(m, H(m) + s)$, onde $H(m) + s$ é a concatenação de $H(m)$ e s . Essa variação é falha? Por quê?
7. O que significa que um documento assinado seja verificável e infalsificável?
8. De que forma o hash da mensagem criptografada com chave pública fornece uma assinatura digital melhor do que a mensagem criptografada com chave pública?
9. Suponha que certifier.com crie um certificado para foo.com. Normalmente, todo o certificado seria criptografado com a chave pública de certifier.com. Verdadeiro ou falso?
10. Suponha que Alice tenha uma mensagem que está pronta para enviar a qualquer pessoa que a solicite. Milhares de pessoas desejam obter a mensagem de Alice, mas cada uma deseja ter certeza da integridade da mensagem. Nesse contexto, você acha que um esquema de integridade baseado em MAC ou em assinatura digital é mais adequado? Por quê?
11. Suponha que Alice e Bob compartilhem duas chaves secretas: uma chave de autenticação S_1 e uma chave de criptografia simétrica S_2 . Amplie a Figura 6 para garantir integridade e confidencialidade.